Connecting a React Frontend to a Database

with Node.js + Express

Student Lab Guide

What You'll Build

A small full-stack app: a React page that displays data pulled live from a MySQL database, through a Node.js/Express API you build yourself.

Stack: React (frontend) → Express (backend API) → MySQL (database)

Before You Start — Checklist

- Node.js and npm installed (run node -v and npm -v to confirm)
- MySQL installed and running locally
- Your existing React app, working, in its own folder
- A code editor (VS Code recommended) and two open terminals

The Big Picture

The browser can't talk directly to a database — it has to go through a server. Express is that server: it receives requests from React, queries the database, and sends back JSON.

```
React (port 3000)  --fetch-->  Express API (port 5000)  --query-->  MySQL Database
```

Step 1: Set Up the Backend Project

Create a server folder next to (NOT inside) your React app's folder:

```
my-project/
├── client/        <- your existing React app
└── server/        <- new Express backend
```

Inside the new server folder, run:

```
mkdir server
cd server
npm init -y
npm install express cors mysql2 dotenv
npm install --save-dev nodemon
```

What each package does:

- express — the web server framework
- cors — allows your React app (different port) to call this API without the browser blocking it
- mysql2 — driver that lets Node talk to MySQL
- dotenv — keeps database credentials out of your source code
- nodemon — restarts the server automatically when you edit code (dev only)

Add these two scripts inside server/package.json:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

Step 2: Store Database Credentials Safely

Create a file named .env inside server/:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=your_database_name
DB_PORT=3306
PORT=5000
```

Create a .gitignore file in server/ so you never commit your password:

```
node_modules
.env
```

✅ **Checkpoint:** You should now have server/.env and server/.gitignore, both filled in with your own values.

Step 3: Connect to the Database

Create server/db.js:

```
const mysql = require('mysql2');
require('dotenv').config();
```

```
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: process.env.DB_PORT,
  waitForConnections: true,
  connectionLimit: 10,
});

module.exports = pool.promise();
```

A connection pool (instead of a single connection) reuses connections efficiently — it's the standard approach for a real app.

If you don't already have a table to work with, create one:

```
CREATE DATABASE your_database_name;

USE your_database_name;

CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  course VARCHAR(100)
);

INSERT INTO students (name, email, course) VALUES
  ('Amina Yusuf', 'amina@example.com', 'Computer Science'),
  ('John Otieno', 'john@example.com', 'Information Technology'),
  ('Grace Wanjiru', 'grace@example.com', 'Software Engineering');
```

Step 4: Build the Express Server

Create server/index.js:

```
const express = require('express');
const cors = require('cors');
require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json());

app.get('/', (req, res) => {
  res.send('API is running');
});

const studentRoutes = require('./routes/students');
app.use('/api/students', studentRoutes);

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

`app.use(cors())` is the piece that lets your React app (running on a different port) successfully call this API. Without it, the browser blocks the request.

Step 5: Query the Database

Create a new folder server/routes/ and inside it, students.js:

```
const express = require('express');
const router = express.Router();
const db = require('../db');

// GET all students
router.get('/', async (req, res) => {
  try {
    const [rows] = await db.query('SELECT * FROM students');
    res.json(rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to fetch students' });
  }
});

module.exports = router;
```

Step 6: Test the Backend on Its Own

Before touching React, confirm the API works by itself. In your terminal:

```
cd server
npm run dev
```

Visit `http://localhost:5000/api/students` in your browser. You should see JSON like:

```
[
  { "id": 1, "name": "Amina Yusuf", "email": "amina@example.com", "course":
"Computer Science" },
  { "id": 2, "name": "John Otieno", "email": "john@example.com", "course":
"Information Technology" }
]
```

✅ **Checkpoint:** Do not move on to Step 7 until this JSON shows up in your browser. If it fails, the problem is in the backend or database — fix it here first.

Step 7: Connect the React Frontend

In your client/ folder, create src/StudentsTable.js:

```
import { useState, useEffect } from 'react';

function StudentsTable() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('http://localhost:5000/api/students')
      .then((res) => {
        if (!res.ok) throw new Error('Network response was not ok');
        return res.json();
      })
      .then((data) => {
        setStudents(data);
        setLoading(false);
      })
      .catch((err) => {
        setError(err.message);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Loading students...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <table border="1" cellPadding="8" style={{ borderCollapse: 'collapse' }}>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Email</th>
          <th>Course</th>
        </tr>
      </thead>
      <tbody>
        {students.map((student) => (
          <tr key={student.id}>
            <td>{student.id}</td>
            <td>{student.name}</td>
            <td>{student.email}</td>
            <td>{student.course}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}

export default StudentsTable;
```

Then render it inside src/App.js:

```
import StudentsTable from './StudentsTable';

function App() {
```

```
  return (
    <div>
      <h1>Students</h1>
      <StudentsTable />
    </div>
  );
}

export default App;
```

Why useEffect? It runs the fetch once when the component mounts, instead of on every render. The empty array [] at the end is what limits it to "run once."

Step 8: Run Both Servers Together

You need two terminals open at the same time:

```
# Terminal 1
cd server
npm run dev

# Terminal 2
cd client
npm start
```

Open `http://localhost:3000` — the table should populate with data pulled live from the database.

✅ **Checkpoint:** If you can see the student table with real data from MySQL rendered in the browser, the lab is complete.

Troubleshooting Guide

Symptom	Likely Cause	Fix
`CORS policy error in browser console`	cors() middleware missing or misconfigured	Confirm app.use(cors()) is in index.js before your routes
`Failed to fetch`	Backend isn't running, or wrong port	Check the server terminal is running; confirm the URL matches PORT in .env
`ECONNREFUSED from Express to MySQL`	MySQL isn't running, or wrong credentials	Start MySQL service; double-check .env values
`Table renders but empty`	Table has no rows, or wrong table name in query	Check the database directly with SELECT * FROM students;
`Cannot GET /api/students`	Route path or method mismatch	Confirm the route path matches exactly, including /api/students

Try It Yourself — Practice Exercises

Once your table is working, attempt these on your own to deepen your understanding:

- Add a new column to the students table (e.g. year_of_study) and display it in the React table.
- Add a search input above the table that filters students by name (client-side, using .filter() on the array in state).
- Create a second table (e.g. courses) and a second API route (/api/courses), then display it on the same page.
- Add a POST route on the backend and a simple form on the frontend to insert a new student.
- Move the hardcoded http://localhost:5000 URL into a .env file in the React app using REACT_APP_API_URL.

Data Flow Summary

- React component mounts → useEffect fires → fetch() call sent to Express
- Express route receives the request → runs a SQL query via the connection pool
- MySQL returns rows → Express sends them back as JSON
- React stores the JSON in state → re-renders the table with real data